

THE FORUM

Manual Técnico

Abel Constantino Muñoz



Índice

1. Introducción.....	4
2. Descripción General del Sistema.....	4
3. Tecnologías Utilizadas.....	5
3.1 Tecnologías Frontend.....	5
3.2 Backend as a Service (Supabase).....	7
3.3 Integración de APIs externas.....	7
3.4 Despliegue y Control de Versiones.....	8
4. Arquitectura del Sistema.....	8
4.1 Capa de Presentación (Frontend / UI).....	8
4.2 Capa de Enrutamiento y Gestión de Estado.....	8
4.3 Capa de Servicios.....	9
4.4 Capa de Persistencia y Backend (Supabase).....	9
4.5 Flujo de Comunicación Típico (Ej. Crear Post).....	10
5. Estructura del Proyecto.....	10
6. Integración de APIs y Lógica de Servidor.....	11
6.1 Consumo de API de RAWG.....	11
6.2 Procedimientos Almacenados (RPC) en Supabase.....	11
7. Entorno, Configuración y Despliegue.....	12
7.1 Variables de Entorno.....	12
7.2 Pipeline de Despliegue en Vercel.....	12
8. Requisitos del Sistema.....	13
8.1 Requisitos Funcionales.....	13
8.2 Requisitos no Funcionales.....	13
9. Base de Datos.....	13
9.1 Entidades Principales y Descripción de Tablas.....	14
9.2 Relaciones entre Entidades.....	15
10. Decisiones Técnicas y Diseño.....	16
11. Gestión de Roles y Permisos.....	17
12. Seguridad.....	17
13. Optimización de Rendimiento.....	18
13.1 Debouncing en Peticiones a APIs.....	18
13.2 CSS Purgado y Optimizado.....	18

14. Sistema de Diseño y Accesibilidad (UI/UX).....	18
15. Manejo de Errores y Prevención de Vulnerabilidades.....	19
15.1 Sanitización de Entradas (XSS).....	19
15.2 Resiliencia de la Interfaz.....	19
16. Mejoras Futuras (Roadmap).....	20
Anexos.....	21
Scripts de SQL usados en supabase.....	21
Esquema de la BBDD.....	21
Add biografia Columns To Usuario.....	24
RLS con roles para foros, posts y notificaciones.....	24
Categorías de los foros.....	26
Row Level Security Policies for Juegos and Foros.....	26
RLS for Public Reads and Authenticated Inserts.....	27
Políticas de Admin para Actualizar y Borrar Usuarios.....	27
Admin ban & profile cleanup.....	28
Policies for Self-Profile and Avatar Storage.....	29
Policies for Public Avatar Storage.....	30
Add Foro Closed Flag & Update Policy.....	30
Generación automática de notificaciones en nuevos posts.....	30

1. Introducción

Este documento describe la arquitectura técnica, las tecnologías utilizadas y la estructura interna del sistema de la aplicación **THE FORUM**. Está dirigido a desarrolladores, evaluadores o personal técnico que necesite comprender el funcionamiento interno del sistema, las decisiones de diseño tomadas durante el desarrollo y las vías de escalabilidad futuras.

2. Descripción General del Sistema

THE FORUM es una plataforma web desarrollada bajo el paradigma de *Single Page Application* (SPA) orientada a la creación, gestión y participación en comunidades y foros temáticos sobre videojuegos. Su objetivo principal es centralizar debates en un entorno moderno, fluido y altamente interactivo.

Para evitar las limitaciones de mantener un catálogo estático y manual, el sistema se nutre de una API externa (RAWG). Esto proporciona una base de datos global y en tiempo real con miles de videojuegos, permitiendo a los usuarios buscar un título específico al instante y abrir nuevos espacios de discusión categorizados en torno a él.

El sistema está diseñado para cubrir de principio a fin el ciclo de vida de una comunidad online, abarcando las siguientes áreas funcionales:

- **Gestión de Usuarios y Perfiles Personalizados:** Un sistema de autenticación seguro que otorga a cada miembro un perfil público editable, donde pueden configurar su avatar, biografía y visualizar su rango dentro de la comunidad.
- **Participación e Interacción:** Creación ágil de hilos de debate y un sistema de lectura secuencial. Incluye funcionalidades avanzadas como respuestas anidadas mediante citas (*quotes*), lo que permite a los usuarios mantener el contexto de la conversación en hilos con una alta participación.
- **Navegación y Filtrado Personalizado:** Interfaces dedicadas, como el panel de "Mis Foros", que otorgan a los usuarios herramientas de búsqueda en tiempo real y filtros por categoría para localizar y gestionar sus propios debates rápidamente sin perderse en el directorio general.

- **Moderación y Control de Acceso (RBAC):** La plataforma implementa un sistema jerárquico de permisos basado en tres roles (Administrador, Moderador y Usuario Estándar). Las herramientas de control de la comunidad permiten:
 - Cierre y reapertura de foros para detener o reanudar la participación en temas específicos.
 - Eliminación de publicaciones de otros usuarios, dotando a los moderadores de las herramientas necesarias para limpiar contenido tóxico, *spam* o inapropiado.
 - Autonomía para el usuario estándar, permitiéndole el borrado seguro de sus propios mensajes.
- **Experiencia de Usuario (UX) Inmersiva:** La arquitectura de la interfaz está diseñada para evitar salidas de contexto. Sustituye las interrupciones nativas del navegador (como ventanas de alerta o confirmación) por diálogos modales integrados en el propio diseño oscuro (*Dark Mode*) de la aplicación, ofreciendo una navegación ininterrumpida y profesional.

3. Tecnologías Utilizadas

Para el desarrollo de THE FORUM se ha optado por un *stack* tecnológico moderno (conocido habitualmente en la industria como Stack React + BaaS), priorizando el rendimiento, la escalabilidad y la experiencia de desarrollo (DX).

3.1 Tecnologías Frontend

React (v18+): Librería principal para la construcción de interfaces de usuario.

- *Justificación:* Su arquitectura basada en componentes reutilizables y la gestión eficiente del DOM virtual (Virtual DOM) encajan perfectamente con la naturaleza dinámica de un foro, donde los componentes (tarjetas de juegos, posts, modales) se repiten constantemente con diferentes datos.

TypeScript: Superset de JavaScript que dota al proyecto de tipado estático estricto.

- *Justificación:* Al manejar estructuras de datos relacionales complejas provenientes de Supabase y de la API de RAWG, TypeScript previene errores

en tiempo de compilación y mejora el autocompletado, reduciendo drásticamente los errores en tiempo de ejecución.

Vite: Entorno de desarrollo y empaquetador (Bundler) de nueva generación.

- *Justificación:* Sustituye a herramientas tradicionales como Webpack (Create React App) ofreciendo tiempos de arranque casi instantáneos gracias a la recarga de módulos en caliente (HMR) nativa del navegador y una compilación de producción ultra optimizada mediante Rollup.

Tailwind CSS: Framework de CSS basado en clases de utilidad (utility-first).

- *Justificación:* Permite un desarrollo de estilos ágil sin necesidad de archivos CSS externos ni cambios de contexto. Su configuración mediante variables hace que el mantenimiento del diseño oscuro (Dark Mode) y la coherencia visual sean altamente eficientes.

shadcn/ui & Lucide React: Colección de componentes de interfaz accesibles y biblioteca de iconos.

- *Justificación:* A diferencia de librerías tradicionales como Bootstrap o Material UI, shadcn/ui no inyecta estilos bloqueantes. El código de los componentes se integra directamente en el proyecto, otorgando control absoluto sobre el diseño, la accesibilidad (WAI-ARIA) y las animaciones sin inflar el peso de la aplicación.

Zustand: Gestor de estado global de la aplicación.

- *Justificación:* Se ha elegido frente a alternativas más pesadas como Redux por su minimalismo y nula necesidad de *boilerplate* (código repetitivo). Se utiliza principalmente para gestionar y persistir la sesión de autenticación (authStore).

React Router v6: Librería para el enrutamiento del lado del cliente.

- *Justificación:* Esencial para construir una Single Page Application (SPA), permitiendo la navegación entre el directorio de foros, perfiles y paneles de administración sin recargar la página, así como la protección de rutas privadas.

3.2 Backend as a Service (Supabase)

Supabase se ha elegido como alternativa Open Source a Firebase, destacando por su uso de bases de datos relacionales nativas frente a las NoSQL.

- **PostgreSQL:** Motor de base de datos relacional de código abierto de nivel empresarial.
 - *Justificación:* Un foro requiere relaciones complejas (Usuarios -> Foros -> Posts -> Citas). El modelo relacional de **SQL** garantiza la integridad referencial y permite realizar consultas con **JOIN** (a través de la API de Supabase) de forma natural y eficiente.
- **Supabase Auth:** Sistema integrado de gestión de identidades y **JWT** (JSON Web Tokens).
 - *Justificación:* Externaliza la complejidad y el riesgo de seguridad de encriptar contraseñas. Proporciona métodos directos para el registro, login y gestión de la sesión del usuario.
- **Row Level Security (RLS):** Políticas de seguridad a nivel de fila directamente en PostgreSQL.
 - *Justificación:* Es el núcleo de seguridad del proyecto. Garantiza que, incluso si la API estuviera expuesta, un usuario solo pueda modificar o borrar sus propios registros (posts, perfil), bloqueando peticiones maliciosas directamente en el motor de base de datos.
- **Supabase Storage:** Almacenamiento de objetos (Buckets).
 - *Justificación:* Utilizado para la carga, almacenamiento y entrega pública de los avatares personalizados de los usuarios mediante CDN.

3.3 Integración de APIs externas

RAWG Video Games Database API: API RESTful que proporciona la mayor base de datos abierta de videojuegos del mundo.

- *Justificación:* En lugar de mantener una base de datos propia de miles de videojuegos, THE FORUM consume esta API para poblar el directorio, obtener portadas de alta calidad y realizar búsquedas en tiempo real, guardando en nuestra base de datos únicamente los juegos sobre los que los usuarios deciden interactuar.

3.4 Despliegue y Control de Versiones

Git & GitHub: Sistema de control de versiones distribuido para mantener el histórico de cambios y salvaguardar el código fuente.

Vercel: Plataforma de despliegue en la nube (Edge Network).

- *Justificación:* Integración continua (CI/CD) nativa con GitHub y soporte de primera clase para proyectos basados en Vite. Al subir un cambio al repositorio, Vercel compila y distribuye automáticamente la aplicación globalmente en cuestión de segundos.

4. Arquitectura del Sistema

THE FORUM no utiliza una arquitectura monolítica tradicional, sino que está diseñado bajo un modelo de **Arquitectura Desacoplada basada en BaaS (Backend as a Service)**. Este enfoque separa completamente la interfaz de usuario de la infraestructura de datos, comunicándose exclusivamente a través de APIs.

El sistema se puede dividir lógica y estructuralmente en las siguientes capas.

4.1 Capa de Presentación (Frontend / UI)

Es la capa con la que interactúa el usuario final. Está desarrollada íntegramente en **React** operando como una SPA (Single Page Application).

- **Componentización:** La interfaz está dividida en componentes reutilizables (botones, modales, tarjetas de foros) apoyados en la librería **shadcn/ui** y renderizados dinámicamente mediante el Virtual DOM.
- **Estilado:** El diseño y la adaptabilidad a dispositivos móviles (Responsive Design) se gestionan mediante **Tailwind CSS**, aplicando clases de utilidad directamente sobre el elemento que queremos estilizar.

4.2 Capa de Enrutamiento y Gestión de Estado

Esta capa actúa como el "cerebro" del lado del cliente, decidiendo qué se muestra y qué datos se mantienen en memoria.

- **Estado Global (Zustand):** Se encarga de mantener la sesión del usuario (authStore). Al iniciar sesión, los datos del perfil y los tokens de autenticación se almacenan aquí, permitiendo que cualquier componente sepa si el usuario es

Admin, Moderador o estándar sin necesidad de pasar *props* continuamente (evitando el *prop drilling*).

- **Enrutamiento (React Router v6):** Gestiona la navegación sin recargar la página web. Implementa protección de rutas (Route Guards) para asegurar que un usuario no autenticado sea redirigido a la pantalla de inicio si intenta acceder a /app.

4.3 Capa de Servicios

Actúa como puente entre la lógica de negocio del frontend y las fuentes de datos externas. El proyecto centraliza las peticiones para mantener un código limpio:

- **Servicio BaaS (lib/supabase.ts):** Instancia el cliente de Supabase. A través de este servicio, los componentes lanzan consultas SQL estructuradas como llamadas de JavaScript (ej. `.from('foros').select('*')`).
- **Servicio API Externa (lib/rawg.ts):** Módulo dedicado exclusivamente a realizar peticiones HTTP (mediante fetch) a la API de RAWG para buscar videojuegos y obtener sus detalles, gestionando las claves de API (API Keys) de forma segura.

4.4 Capa de Persistencia y Backend (Supabase)

Esta capa reside en los servidores de Supabase y maneja toda la persistencia y validación final de la aplicación:

- **Base de Datos (PostgreSQL):** Almacena el esquema relacional. Supabase expone esta base de datos automáticamente a través de una API RESTful (PostgREST), permitiendo al frontend interactuar con ella sin necesidad de desarrollar un backend intermedio en Node.js o PHP.
- **Seguridad de Datos (RLS - Row Level Security):** Constituye el cortafuegos principal. Aunque el frontend envíe una petición de borrado de un post, es esta capa la que verifica el **JWT** del usuario y valida mediante políticas **SQL** si el ID del usuario coincide con el **autor_id** del post, o si tiene un **rol_id** de administrador.
- **Almacenamiento (Storage):** Gestiona la subida de archivos binarios (imágenes de avatar) distribuyéndolos a través de una CDN (Content Delivery Network).

4.5 Flujo de Comunicación Típico (Ej. Crear Post)

Para ilustrar la arquitectura, el ciclo de vida de una petición sigue este orden:

1. **Acción del usuario:** El usuario escribe un texto en **ForoDetalle.tsx** y pulsa "Enviar".
2. **Validación en Cliente:** React verifica que el texto no esté vacío y que el estado global de Zustand confirme que hay un usuario logueado.
3. **Petición al Servicio:** El cliente de Supabase envía una petición HTTP POST a la API REST de la base de datos, adjuntando en la cabecera el token JWT de la sesión.
4. **Validación en Servidor:** PostgreSQL recibe la petición, lee el JWT, comprueba las políticas de RLS y, si está autorizado, inserta la fila en la tabla posts.
5. **Respuesta y Actualización:** El servidor devuelve un código de éxito (HTTP 20X). React recibe la respuesta y actualiza el estado local, renderizando el nuevo mensaje instantáneamente en la pantalla del usuario.

5. Estructura del Proyecto

Para garantizar la escalabilidad y mantenibilidad del código, THE FORUM sigue una arquitectura de carpetas modular basada en las mejores prácticas de React. La separación de responsabilidades permite que nuevos desarrolladores puedan entender la topología del sistema rápidamente.

- **src/:** Directorio raíz del código fuente de la aplicación.
 - **assets/:** Archivos estáticos como imágenes, iconos locales o fuentes tipográficas.
 - **components/:** Componentes de interfaz reutilizables.
 - **ui/:** Componentes base genéricos (Botones, Inputs, Modales) generados mediante shadcn/ui.
 - **foros/:** Componentes específicos del dominio de foros (ej. **CrearForoModal.tsx**).
 - **hooks/:** Custom Hooks de React para encapsular lógica compleja y reutilizable (ej. gestión de modo oscuro o *fetching* de datos).
 - **lib/:** Archivos de configuración e instanciación de servicios externos.
 - **supabase.ts:** Inicialización del cliente de Supabase.
 - **rawg.ts:** Funciones de llamada a la API de RAWG.

- **pages/**: Componentes principales que actúan como vistas o pantallas (ej. **Main.tsx**, **ForoDetalle.tsx**, **AdminUsuarios.tsx**, **ForosUsuario.tsx**).
- **store/**: Gestión del estado global de la aplicación.
 - **authStore.ts**: Lógica de sesión con Zustand.
- **types/**: Definiciones de interfaces y tipos de TypeScript para asegurar la consistencia de los datos en toda la aplicación.
- **App.tsx / router.tsx**: Configuración principal de las rutas de la aplicación mediante React Router.
- **index.css**: Archivo de estilos globales donde se definen las variables CSS para el sistema de diseño de Tailwind.

6. Integración de APIs y Lógica de Servidor

6.1 Consumo de API de RAWG

Para nutrir el directorio de videojuegos sin sobrecargar la base de datos propia, THE FORUM se conecta a la API REST pública de RAWG. Las llamadas se centralizan en el servicio **rawg.ts** utilizando peticiones asíncronas (**fetch**).

- **Búsqueda (Endpoints)**: Se consulta el endpoint **/games?search={termino}** para obtener coincidencias en tiempo real cuando el usuario teclea en el buscador de creación de foros.
- **Almacenamiento en caché**: Cuando un usuario crea un foro sobre un juego, el sistema guarda un registro básico en la tabla **videojuegos** de Supabase (ID, Nombre, Portada) para reducir futuras llamadas a RAWG y optimizar los tiempos de carga de la aplicación.

6.2 Procedimientos Almacenados (RPC) en Supabase

Cierta lógica de negocio crítica no debe delegarse al frontend por cuestiones de seguridad y rendimiento. Para ello, se han implementado Procedimientos Almacenados (Remote Procedure Calls) ejecutados directamente en PostgreSQL.

- **banear_y_limpiar_usuario**: Función SQL diseñada para la moderación de la comunidad. En lugar de ejecutar múltiples consultas desde el cliente (lo que causaría latencia y posibles bloqueos), este RPC recibe el ID de un usuario problemático, elimina su perfil de la base de datos pública y actualiza todos sus

posts para “anonimizarlos” (por ejemplo, cambiando el autor a "anónimo" o anulando su ID), garantizando la integridad de los hilos de debate existentes.

7. Entorno, Configuración y Despliegue

7.1 Variables de Entorno

El sistema requiere la configuración de variables de entorno para asegurar las credenciales de los servicios externos. Estas claves nunca se exponen en el repositorio de código fuente:

- **VITE_SUPABASE_URL**: URL del proyecto asignado en Supabase.
- **VITE_SUPABASE_ANON_KEY**: Clave pública para realizar peticiones autenticadas a la base de datos.
- **VITE_RAWG_API_KEY**: Clave de acceso a la API de videojuegos de RAWG.

7.2 Pipeline de Despliegue en Vercel

El despliegue de THE FORUM está automatizado mediante Vercel, ofreciendo un entorno de Integración Continua (CI):

1. **Commit / Push**: El desarrollador sube los cambios a la rama principal (main) en GitHub.
2. **Build**: Vercel detecta el cambio, clona el repositorio, instala las dependencias (npm install) y ejecuta el comando de compilación de Vite (npm run build). Vite empaqueta el código React y TypeScript en archivos HTML, CSS y JavaScript estáticos y minificados.
3. **Distribución Edge**: Los archivos generados se distribuyen globalmente a través de la red Edge de Vercel, garantizando latencias mínimas para los usuarios sin importar su ubicación geográfica.
4. **Gestión de Rutas SPA**: Se configura un archivo vercel.json (o reglas de *rewrites*) para garantizar que todas las peticiones a rutas internas de React Router apunten al archivo index.html, evitando errores 404 al recargar la página.

8. Requisitos del Sistema

8.1 Requisitos Funcionales

RF1: Registro, inicio de sesión y gestión de perfil (cambio de avatar, biografía y nombre).

RF2: Directorio dinámico de videojuegos consumido mediante API externa con buscador en tiempo real.

RF3: Creación de foros asociados a un videojuego y a una categoría.

RF4: Publicación de mensajes dentro de un foro, incluyendo la funcionalidad de citar mensajes anteriores.

RF5: Panel de administración para cambiar roles (Admin, Moderador, Usuario).

RF6: Herramientas de moderación: cierre/reapertura de hilos y eliminación (baneo) de usuarios o borrado de posts individuales.

8.2 Requisitos no Funcionales

Diseño estético coherente: Implementación de un diseño "Gamer/Dark Mode" unificado y responsivo.

Seguridad: Accesos protegidos por rutas privadas y validaciones tanto en cliente como en base de datos.

Feedback visual: Sustitución de alertas nativas del navegador por componentes integrados (Diálogos de confirmación y mensajes de error in-line).

9. Base de Datos

El sistema se basa en un modelo relacional en PostgreSQL gestionado a través de Supabase. La autenticación y credenciales (email y contraseña) se manejan de forma segura en el esquema interno **auth.users** de Supabase, vinculándose mediante clave foránea a la tabla pública de perfiles (**usuarios**).

9.1 Entidades Principales y Descripción de Tablas

usuarios: Contiene los datos públicos del perfil.

- **id** (PK, UUID) -> Referencia a **auth.users.id**
- **username** (Texto, Único)
- **avatar_url** (Texto, URL de la imagen)
- **biografia** (Texto)
- **rol_id** (FK a **roles**)
- **created_at** (Timestamp)

roles: Define los niveles de acceso del sistema.

- **id** (PK, Entero)
- **nombre** (Texto, Único)
- **descripcion** (Texto)

categorias: Clasificación temática de los foros.

- **id** (PK, Entero)
- **nombre** (Texto)
- **descripcion** (Texto)
- **created_at** (Timestamp)

videojuegos: Caché local de la API externa (RAWG) para optimizar consultas.

- **id** (PK, Entero, coincide con ID de RAWG)
- **nombre** (Texto)
- **genero** (Texto)
- **desarrollador** (Texto)
- **fecha_salida** (Date)
- **created_at** (Timestamp)

foros: Hilos de debate principales.

- **id** (PK, UUID)
- **titulo** (Texto)
- **descripcion** (Texto)

- **videojuego_id** (FK a **videojuegos**)
- **categoria_id** (FK a **categorias**)
- **creador_id** (FK a **usuarios**)
- **esta_abierto** / **cerrado** (Booleanos para control de moderación)
- **created_at** (Timestamp)

Posts: Mensajes dentro de cada foro.

- **id** (PK, UUID)
- **foro_id** (FK a **foros**)
- **autor_id** (FK a **usuarios**)
- **contenido** (Texto)
- **post_citado_id** (FK a **posts**, permite anidación/citas)
- **created_at** (Timestamp)

Notificaciones: Registro de alertas para los usuarios.

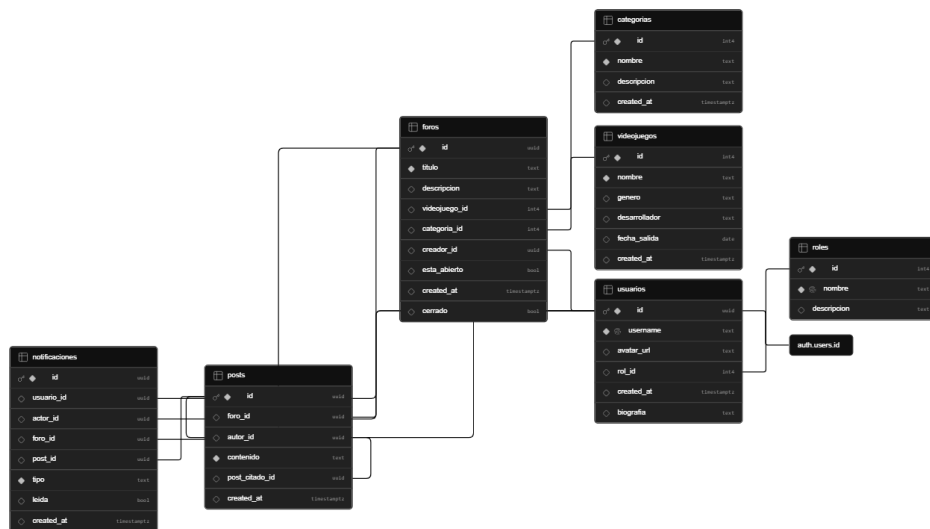
- **id** (PK, UUID)
- **usuario_id** (FK a **usuarios**, el receptor)
- **actor_id** (FK a **usuarios**, quien genera la acción)
- **foro_id** (FK a **foros**)
- **post_id** (FK a **posts**)
- **tipo** (Texto, ej: "respuesta", "mencion")
- **leida** (Booleano)
- **created_at** (Timestamp)

9.2 Relaciones entre Entidades

La arquitectura de la base de datos establece las siguientes cardinalidades:

- **auth.users 1 → 1 usuarios:** Cada cuenta segura de Supabase tiene un único perfil público.
- **roles 1 → N usuarios:** Un rol puede ser asignado a múltiples usuarios.
- **categorias 1 → N foros:** Una categoría agrupa múltiples debates.
- **videojuegos 1 → N foros:** Un videojuego puede tener múltiples foros asociados.
- **usuarios 1 → N foros:** Un usuario puede ser el **creador_id** de varios hilos.

- **foros 1 → N posts:** Un foro contiene múltiples respuestas.
- **usuarios 1 → N posts:** Un usuario puede ser el **autor_id** de varios mensajes.
- **posts 1 → N posts:** Relación autorreferencial (**post_citado_id**) para el sistema de citas.
- **usuarios 1 → N notificaciones:** Un usuario recibe múltiples avisos (**usuario_id**) generados por otros usuarios (**actor_id**).



10. Decisiones Técnicas y Diseño

Durante el ciclo de vida del proyecto (desarrollado hasta mayo de 2026), se tomaron decisiones pivotaes para asegurar la viabilidad comercial y la calidad de la experiencia de usuario (UX):

1. **Cambio de paradigma visual (Chat vs. Foro):** Inicialmente, la visualización de los hilos se planteó con una estética similar a una aplicación de mensajería instantánea (estilo WhatsApp). Tras pruebas de usabilidad, se modificó a un listado secuencial tradicional (estilo foro clásico), lo que mejora drásticamente la legibilidad de textos largos y la visualización jerárquica de las citas entre usuarios.

2. **Centralización de la navegación ("Mis Foros"):** Para evitar que el usuario perdiera el rastro de los hilos creados por él mismo entre el vasto directorio de juegos, se desarrolló una vista dedicada ("Mis Foros") que realiza un filtrado automático en base de datos, agilizando el seguimiento de las comunidades de interés.
3. **Erradicación de interrupciones nativas:** Se invirtió esfuerzo en sustituir todas las funciones nativas de confirmación del navegador (**window.confirm**, **alert**) por modales propios (**ConfirmDialog**), manteniendo la inmersión del usuario y la coherencia del diseño oscuro.
4. **Redirección fluida:** Se programó el comportamiento de la creación de foros para que, una vez la base de datos confirme la inserción, el enrutador redirija automáticamente al usuario al interior del hilo, incentivando la creación inmediata del primer post.

11. Gestión de Roles y Permisos

El sistema contempla un control de acceso basado en tres niveles:

1. **Admin:** Acceso al panel de administración. Capacidad para banear usuarios (anonimizando sus posts mediante RPC en base de datos), cambiar roles, cerrar/abrir foros y borrar cualquier post.
2. **Moderador :** Capacidad para cerrar/abrir foros y borrar posts tóxicos para mantener la comunidad limpia.
3. **Usuario Estándar:** Capacidad para crear foros, comentar, editar su perfil y borrar únicamente sus propios mensajes.

12. Seguridad

La autenticación delega la gestión de contraseñas y tokens JWT al proveedor seguro de Supabase.

Uso de **Row Level Security (RLS)** en PostgreSQL para garantizar que un usuario modificado maliciosamente desde el frontend no pueda alterar datos de otros usuarios directamente en la base de datos.

Protección de rutas de React para evitar la navegación a vistas administrativas sin el estado correspondiente en **authStore**.

13. Optimización de Rendimiento

Durante el desarrollo de THE FORUM se han aplicado diversas técnicas de optimización para garantizar que la aplicación sea rápida y consuma los mínimos recursos posibles en el dispositivo del usuario final.

13.1 Debouncing en Peticiones a APIs

En la funcionalidad de "Crear Nuevo Foro", el usuario dispone de un buscador en tiempo real de videojuegos. Para evitar saturar la API de RAWG con una petición HTTP por cada tecla pulsada, se ha implementado un mecanismo de *Debounce* (retraso intencionado) mediante un `setTimeout` de 500 milisegundos en React. El sistema solo dispara la búsqueda a la base de datos externa cuando el usuario deja de escribir durante medio segundo, reduciendo drásticamente el tráfico de red y mejorando la fluidez del formulario.

13.2 CSS Purgado y Optimizado

Gracias al uso de Tailwind CSS integrado con Vite, el empaquetador escanea todos los archivos `.tsx` del proyecto durante el proceso de *build*. Cualquier clase CSS que no se haya utilizado explícitamente en el código es purgada y eliminada del archivo final. Esto da como resultado una hoja de estilos (CSS) de unos pocos kilobytes, garantizando un tiempo de renderizado inicial ultrarrápido (FCP - First Contentful Paint).

14. Sistema de Diseño y Accesibilidad (UI/UX)

La interfaz de usuario no solo se ha diseñado bajo criterios estéticos, sino siguiendo normativas de usabilidad y consistencia visual.

- **Tipografía:** Se ha integrado la fuente *Geist Variable* como tipografía principal. Al ser una fuente variable, permite múltiples pesos tipográficos (desde ultra-fina hasta extra-negrita) utilizando un único archivo fuente, optimizando los tiempos de carga y mejorando la legibilidad en pantallas de alta densidad de píxeles.
- **Paleta de Colores (Design Tokens):** El sistema utiliza variables CSS globales (`--background`, `--primary`, `--foreground`) inyectadas en el `:root` del archivo `index.css`. La paleta principal se basa en tonos *Slate* (grises azulados) para los fondos y *Violet* (violetas vibrantes) para los elementos de acción principal, reduciendo la fatiga visual del usuario en sesiones prolongadas.
- **Feedback Visual Inmersivo:** Se ha erradicado el uso de ventanas emergentes bloqueantes nativas del navegador (`window.alert` o `window.confirm`). Todo el manejo de errores (como fallos en la actualización del perfil o nombres de usuario duplicados) se muestra directamente en la interfaz mediante mensajes de texto integrados en los formularios, y las acciones destructivas (como borrar un post) utilizan un componente propio (`ConfirmDialog`) que mantiene la estética y oscurece el fondo mediante técnicas de desenfoque (`backdrop-blur`).

15. Manejo de Errores y Prevención de Vulnerabilidades

La aplicación implementa múltiples capas de seguridad y validación:

15.1 Sanitización de Entradas (XSS)

Al utilizar React, cualquier texto introducido por un usuario (por ejemplo, el contenido de un post o la biografía de su perfil) es automáticamente sanitizado y escapado antes de ser renderizado en el DOM. Esto previene ataques de *Cross-Site Scripting* (XSS), impidiendo que un atacante pueda inyectar código JavaScript malicioso en los hilos de debate.

15.2 Resiliencia de la Interfaz

Las llamadas asíncronas a la base de datos están encapsuladas en bloques `try/catch`. Junto a esto, se hace un uso extensivo de estados de carga (`isLoading` o `cargando`). Cuando un componente está esperando datos del servidor (por ejemplo, al abrir

ForoDetalle.tsx), se muestran animaciones de esqueleto o mensajes de carga (**animate-pulse**) en lugar de interfaces rotas o en blanco, manteniendo informado al usuario sobre el estado del sistema en todo momento.

16. Mejoras Futuras (Roadmap)

La arquitectura actual sienta las bases para futuras actualizaciones que aumenten la retención de usuarios (engagement) y la accesibilidad de la plataforma:

1. **Modo Claro / Tema Personalizable:** Implementación completa del "Light Mode" invirtiendo las variables globales de Tailwind CSS, permitiendo al usuario alternar entre modos visuales y guardando su preferencia en el almacenamiento local.
2. **Internacionalización y Traducción mediante IA:** Soporte multi-idioma (i18n) para la interfaz base del foro y, como innovación clave, un sistema de traducción automática de los posts utilizando Inteligencia Artificial, permitiendo que jugadores de todo el mundo debatan rompiendo la barrera del idioma.
3. **Sistema de Comunidades:** Agrupación de usuarios en clanes o hermandades privadas con foros de acceso restringido.
4. **Gamificación (Sistema de puntos):** Otorgar experiencia y moneda virtual a los usuarios por su participación activa (crear posts de valor, recibir interacciones).
5. **Tienda Virtual:** Un espacio donde los usuarios puedan gastar los puntos obtenidos en mejoras cosméticas para su perfil, como marcos para los avatares, colores de nombre o insignias especiales visibles en sus posts.
6. **Minijuegos 1vs1:** Integración de pequeños retos web entre usuarios del foro para apostar o ganar puntos adicionales, aumentando el tiempo de sesión en la plataforma.
7. **Refuerzo de Seguridad y Anti-Spam:** Implementación de *Rate Limiting* (límite de peticiones por minuto) en el backend para la creación de posts, evitando ataques de sobrecarga, bots o comportamiento automatizado perjudicial.
8. **Mejora del Sistema Social:** Permitir a los usuarios visitar los perfiles de los demás usuarios y crear un sistema de amistades
9. **Mensajería Privada:** Implementación de chats privados para comunicarte con los demás usuarios amigos o no

Anexos

Scripts de SQL usados en supabase

Esquema de la BBDD

-- 1. TABLA ROLES

```
CREATE TABLE public.roles (  
  id SERIAL PRIMARY KEY,  
  nombre TEXT UNIQUE NOT NULL,  
  descripcion TEXT  
);
```

-- Insertamos los 3 roles por defecto

```
INSERT INTO public.roles (nombre, descripcion) VALUES  
( 'admin', 'Administrador total del sistema'),  
( 'moderador', 'Puede abrir/cerrar foros y moderar posts'),  
( 'usuario', 'Usuario estándar registrado');
```

-- 2. TABLA USUARIOS (Perfil público)

-- Se enlaza con auth.users de Supabase mediante el campo "id"

```
CREATE TABLE public.usuarios (  
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  username TEXT UNIQUE NOT NULL,  
  avatar_url TEXT,  
  rol_id INTEGER REFERENCES public.roles(id) DEFAULT 3, -- Por defecto  
  'usuario' (ID 3)  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- 3. TABLA VIDEOJUEGO

-- Usaremos el ID de la API de RAWG como ID primario aquí (por eso es INTEGER y no UUID)

```
CREATE TABLE public.videojuegos (  
  id INTEGER PRIMARY KEY,  
  nombre TEXT NOT NULL,  
  genero TEXT,  
  desarrollador TEXT,
```

```
    fecha_salida DATE,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- 4. TABLA CATEGORIA

```
CREATE TABLE public.categorias (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT NOT NULL,  
    descripcion TEXT,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- Insertamos un par de categorías de ejemplo

```
INSERT INTO public.categorias (nombre, descripcion) VALUES  
( 'Guías y Builds', 'Comparte tus mejores estrategias'),  
( 'Soporte Técnico', 'Ayuda con bugs, crasheos y rendimiento'),  
( 'Discusión General', 'Habla sobre la historia, curiosidades y más');
```

-- 5. TABLA FORO

```
CREATE TABLE public.foros (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    titulo TEXT NOT NULL,  
    descripcion TEXT,  
    videojuego_id INTEGER REFERENCES public.videojuegos(id) ON DELETE  
CASCADE,  
    categoria_id INTEGER REFERENCES public.categorias(id) ON DELETE SET  
NULL,  
    creador_id UUID REFERENCES public.usuarios(id) ON DELETE CASCADE,  
    esta_abierto BOOLEAN DEFAULT TRUE, -- Para que admin/mod puedan  
cerrarlos  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- 6. TABLA POST

```
CREATE TABLE public.posts (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    foro_id UUID REFERENCES public.foros(id) ON DELETE CASCADE,  
    autor_id UUID REFERENCES public.usuarios(id) ON DELETE CASCADE,
```

```
    contenido TEXT NOT NULL,  
    post_citado_id UUID REFERENCES public.posts(id) ON DELETE SET NULL, --  
    ¡La clave para citar posts!  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- 7. TABLA NOTIFICACIONES

```
CREATE TABLE public.notificaciones (  
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
    usuario_id UUID REFERENCES public.usuarios(id) ON DELETE CASCADE, --  
    Quien recibe la notificación  
    actor_id UUID REFERENCES public.usuarios(id) ON DELETE CASCADE, --  
    Quien hizo la acción (respondió/citó)  
    foro_id UUID REFERENCES public.foros(id) ON DELETE CASCADE,  
    post_id UUID REFERENCES public.posts(id) ON DELETE CASCADE,  
    tipo TEXT NOT NULL, -- Ej: 'cita', 'nuevo_post_en_tu_foro'  
    leida BOOLEAN DEFAULT FALSE,  
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- FUNCIÓN Y TRIGGER PARA CREAR USUARIO AUTOMÁTICAMENTE

```
-- Esta función se ejecuta cada vez que alguien se registra en Supabase Auth  
CREATE OR REPLACE FUNCTION public.handle_new_user()  
RETURNS TRIGGER AS $$  
BEGIN  
    INSERT INTO public.usuarios (id, username, rol_id)  
    VALUES (  
        NEW.id,  
        -- Aquí asume que en React, al registrar al usuario, le mandas el 'username' en la  
        metadata  
        COALESCE(NEW.raw_user_meta_data->>'username', 'User_' ||  
        substr(NEW.id::text, 1, 8)),  
        3 -- ID 3 = 'usuario'  
    );  
    RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- El trigger que dispara la función anterior
CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE PROCEDURE public.handle_new_user();
```

Add biografia Columns To Usuario

```
ALTER TABLE public.usuarios
ADD COLUMN biografia TEXT;
```

RLS con roles para foros, posts y notificaciones

```
-----
-- 1. FUNCIÓN AUXILIAR PARA OBTENER EL ROL DEL USUARIO
-----
```

```
-- Esta función nos dirá de forma segura si el que hace la petición es admin(1), mod(2)
o usuario(3)
CREATE OR REPLACE FUNCTION public.get_rol_usuario()
RETURNS integer AS $$
  SELECT rol_id FROM public.usuarios WHERE id = auth.uid();
$$ LANGUAGE sql SECURITY DEFINER;
```

```
-----
-- 2. ACTIVAR RLS EN LAS TABLAS
-----
```

```
ALTER TABLE public.usuarios ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.foros ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.posts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.notificaciones ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.categorias ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.videojuegos ENABLE ROW LEVEL SECURITY;
```

```
-----
-- 3. POLÍTICAS DE SEGURIDAD (RLS)
-----
```

```
-- TABLA USUARIOS
```



```
-- Todos pueden ver los perfiles
CREATE POLICY "Perfiles visibles para todos" ON public.usuarios FOR SELECT
USING (true);
-- Solo tú puedes editar tu propio perfil (o un admin)
CREATE POLICY "Editar propio perfil o admin" ON public.usuarios FOR UPDATE
USING (auth.uid() = id OR public.get_rol_usuario() = 1);

-- TABLAS DE SOLO LECTURA (Categorías y Videojuegos)
-- La app necesita leerlas, pero solo se escribirán desde el panel o por fetch interno
CREATE POLICY "Categorías visibles" ON public.categorias FOR SELECT USING
(true);
CREATE POLICY "Videojuegos visibles" ON public.videojuegos FOR SELECT
USING (true);

-- TABLA FOROS
-- Todos pueden ver los foros
CREATE POLICY "Foros visibles para todos" ON public.foros FOR SELECT
USING (true);
-- Cualquier usuario logueado puede crear un foro
CREATE POLICY "Usuarios crean foros" ON public.foros FOR INSERT WITH
CHECK (auth.uid() = creador_id);
-- Para editar (ej: cerrar el foro), puede hacerlo el creador, un admin(1) o un mod(2)
CREATE POLICY "Editar foros" ON public.foros FOR UPDATE
USING (auth.uid() = creador_id OR public.get_rol_usuario() IN (1, 2));

-- TABLA POSTS
-- Todos pueden ver los posts
CREATE POLICY "Posts visibles" ON public.posts FOR SELECT USING (true);
-- Cualquier usuario logueado puede escribir un post
CREATE POLICY "Usuarios crean posts" ON public.posts FOR INSERT WITH
CHECK (auth.uid() = autor_id);
-- Solo el autor, o los admins/mods pueden editar/borrar un post
CREATE POLICY "Editar posts" ON public.posts FOR UPDATE
USING (auth.uid() = autor_id OR public.get_rol_usuario() IN (1, 2));
CREATE POLICY "Borrar posts" ON public.posts FOR DELETE
USING (auth.uid() = autor_id OR public.get_rol_usuario() IN (1, 2));

-- TABLA NOTIFICACIONES
```

```
-- Solo ves TUS notificaciones
CREATE POLICY "Ver mis notificaciones" ON public.notificaciones FOR SELECT
USING (auth.uid() = usuario_id);
-- El sistema (y otros usuarios al comentar) pueden generar notificaciones
CREATE POLICY "Crear notificaciones" ON public.notificaciones FOR INSERT
WITH CHECK (true);
-- Solo tú puedes actualizar tus notificaciones (para marcarlas como leídas)
CREATE POLICY "Actualizar mis notificaciones" ON public.notificaciones FOR
UPDATE USING (auth.uid() = usuario_id);
```

Categorías de los foros

```
INSERT INTO public.categorias (nombre, descripcion) VALUES
('Noticias y Anuncios', 'Novedades sobre parches, actualizaciones y eventos'),
('Multimedia', 'Comparte tus mejores clips, capturas de pantalla y fan-art'),
('Reclutamiento y Clanes', 'Busca gente para jugar o promociona tu clan'),
('Off-Topic', 'Para hablar de cualquier cosa no relacionada con videojuegos');
```

Row Level Security Policies for Juegos and Foros

```
-- 1. Volvemos a activar el RLS por si lo habías desactivado
alter table public.videojuegos enable row level security;
alter table public.foros enable row level security;

-- 2. POLÍTICAS PARA LA TABLA VIDEOJUEGOS
-- Permitimos que cualquiera vea los juegos
create policy "Cualquiera puede ver juegos"
on public.videojuegos for select
using (true);

-- Permitimos que solo los usuarios logueados guarden juegos nuevos
create policy "Logueados pueden insertar juegos"
on public.videojuegos for insert
to authenticated
with check (true);

-- 3. POLÍTICAS PARA LA TABLA FOROS
-- Permitimos que cualquiera vea los foros
create policy "Cualquiera puede ver foros"
```

```
on public.foros for select  
using (true);
```

```
-- Permitimos que usuarios logueados creen foros,  
-- PERO solo si se ponen a sí mismos como creador_id (seguridad extra)  
create policy "Logueados pueden crear foros"  
on public.foros for insert  
to authenticated  
with check (auth.uid() = creador_id);
```

RLS for Public Reads and Authenticated Inserts

```
-- 1. Asegurarnos de que el RLS está activado  
alter table public.posts enable row level security;
```

```
-- 2. Permitir que cualquiera pueda leer los mensajes (posts)  
create policy "Cualquiera puede leer posts"  
on public.posts for select  
using (true);
```

```
-- 3. Permitir escribir SOLO a usuarios logueados y comprobando su identidad  
create policy "Logueados pueden insertar posts"  
on public.posts for insert  
to authenticated  
with check (auth.uid() = autor_id);
```

Políticas de Admin para Actualizar y Borrar Usuarios

```
-- Permitir que los administradores (rol_id = 1) hagan UPDATE en cualquier fila  
CREATE POLICY "Admins pueden actualizar cualquier usuario"  
ON public.usuarios FOR UPDATE  
TO authenticated  
USING (  
  EXISTS (  
    SELECT 1 FROM public.usuarios  
    WHERE id = auth.uid() AND rol_id = 1  
  )  
);
```

```
-- Permitir que los administradores borren cualquier fila
CREATE POLICY "Admins pueden borrar cualquier usuario"
ON public.usuarios FOR DELETE
TO authenticated
USING (
    EXISTS (
        SELECT 1 FROM public.usuarios
        WHERE id = auth.uid() AND rol_id = 1
    )
);
```

Admin ban & profile cleanup

```
ALTER TABLE public.posts DROP CONSTRAINT IF EXISTS posts_autor_id_fkey;
ALTER TABLE public.posts ADD CONSTRAINT posts_autor_id_fkey
    FOREIGN KEY (autor_id) REFERENCES public.usuarios(id) ON DELETE SET
NULL;
```

```
ALTER TABLE public.foros DROP CONSTRAINT IF EXISTS
foros_creador_id_fkey;
ALTER TABLE public.foros ADD CONSTRAINT foros_creador_id_fkey
    FOREIGN KEY (creador_id) REFERENCES public.usuarios(id) ON DELETE SET
NULL;
```

```
-- 2. Nueva función de baneo y limpieza de perfil
CREATE OR REPLACE FUNCTION public.banear_y_limpiar_usuario(target_id
uuid)
RETURNS void AS $$
BEGIN
    -- Verificar que el ejecutor sea Admin
    IF EXISTS (SELECT 1 FROM public.usuarios WHERE id = auth.uid() AND rol_id
= 1) THEN

        -- A) Bloqueamos el acceso en auth.users (ponemos baneo infinito)
        UPDATE auth.users
        SET banned_until = 'infinity'
        WHERE id = target_id;
```

```
-- B) Borrarnos el perfil público
-- Esto hará que autor_id en posts pase a ser NULL automáticamente
DELETE FROM public.usuarios WHERE id = target_id;

ELSE
    RAISE EXCEPTION 'Acceso denegado: Se requieren permisos de administrador.';
END IF;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Policies for Self-Profile and Avatar Storage

```
-- 1. PERMISO PARA LA TABLA USUARIOS:
-- Permitir que cada usuario pueda actualizar SOLO su propia fila (auth.uid() = id)
CREATE POLICY "Usuarios pueden editar su propio perfil"
ON public.usuarios
FOR UPDATE
TO authenticated
USING (auth.uid() = id);

-- 2. PERMISOS PARA EL STORAGE (Subir fotos):
-- Permitir a cualquier usuario logueado subir (INSERT) una foto al bucket 'avatars'
CREATE POLICY "Usuarios pueden subir fotos"
ON storage.objects
FOR INSERT
TO authenticated
WITH CHECK (bucket_id = 'avatars');

-- 3. PERMISOS PARA EL STORAGE (Sobreescribir fotos):
-- Permitir actualizar (UPDATE) la foto si ya habían subido una antes
CREATE POLICY "Usuarios pueden actualizar fotos"
ON storage.objects
FOR UPDATE
TO authenticated
USING (bucket_id = 'avatars');
```

Policies for Public Avatar Storage

```
-- 1. Permitir que TODO EL MUNDO (incluso sin loguear) pueda VER las fotos
```

```
-- (Necesario para que las imágenes carguen en la app y para que el upsert funcione)
CREATE POLICY "Lectura publica de avatares"
ON storage.objects FOR SELECT
USING (bucket_id = 'avatars');
```

```
-- 2. Permitir a los usuarios LOGUEADOS hacer TODO (Subir, Actualizar, Borrar) en
ese bucket
CREATE POLICY "Usuarios logueados pueden modificar avatares"
ON storage.objects FOR ALL
TO authenticated
USING (bucket_id = 'avatars')
WITH CHECK (bucket_id = 'avatars');
```

Add Foro Closed Flag & Update Policy

```
-- 1. Añadimos la columna 'cerrado' a la tabla foros (por defecto en falso/abierto)
ALTER TABLE public.foros ADD COLUMN IF NOT EXISTS cerrado BOOLEAN
DEFAULT false;
```

```
-- 2. Permitimos que Admins (1) y Moderadores (2) puedan actualizar la tabla foros
CREATE POLICY "Mods y Admins pueden actualizar foros"
ON public.foros
FOR UPDATE
TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.usuarios
    WHERE id = auth.uid() AND rol_id IN (1, 2)
  )
);
```

Generación automática de notificaciones en nuevos posts

```
-- 1. Función que evalúa cada nuevo post
CREATE OR REPLACE FUNCTION public.generar_notificaciones()
RETURNS TRIGGER AS $$
DECLARE
  v_creador_foro uuid;
  v_autor_citado uuid;
BEGIN
```

-- A. NOTIFICACIÓN AL CREADOR DEL FORO (Si alguien responde en su tema)

```
SELECT creador_id INTO v_creador_foro FROM public.foros WHERE id =  
NEW.foro_id;
```

-- Verificamos que el creador del foro exista y no sea el mismo que está respondiendo

```
IF v_creador_foro IS NOT NULL AND v_creador_foro != NEW.autor_id THEN  
    INSERT INTO public.notificaciones (usuario_id, actor_id, foro_id, post_id, tipo)  
    VALUES (v_creador_foro, NEW.autor_id, NEW.foro_id, NEW.id,  
'respuesta_foro');  
END IF;
```

-- B. NOTIFICACIÓN POR CITA (Si el post cita a otro usuario)

```
IF NEW.post_citado_id IS NOT NULL THEN  
    SELECT autor_id INTO v_autor_citado FROM public.posts WHERE id =  
NEW.post_citado_id;
```

-- Verificamos que el citado exista y no se esté auto-citando

```
IF v_autor_citado IS NOT NULL AND v_autor_citado != NEW.autor_id THEN  
    INSERT INTO public.notificaciones (usuario_id, actor_id, foro_id, post_id,  
tipo)  
    VALUES (v_autor_citado, NEW.autor_id, NEW.foro_id, NEW.id, 'cita');  
END IF;  
END IF;
```

```
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

-- 2. Conectar la función a la tabla posts (se ejecuta SIEMPRE que haya un INSERT)

```
DROP TRIGGER IF EXISTS trigger_notificaciones_post ON public.posts;  
CREATE TRIGGER trigger_notificaciones_post  
AFTER INSERT ON public.posts  
FOR EACH ROW EXECUTE FUNCTION public.generar_notificaciones();
```

-- 3. Dar permisos para que los usuarios puedan ver y marcar como leídas sus notificaciones

```
CREATE POLICY "Usuarios pueden ver sus notificaciones"  
ON public.notificaciones FOR SELECT TO authenticated USING (usuario_id =  
auth.uid());
```

```
CREATE POLICY "Usuarios pueden actualizar sus notificaciones"  
ON public.notificaciones FOR UPDATE TO authenticated USING (usuario_id =  
auth.uid());
```